How the spaceplane autopilot works

krpcLab · spaceplane · what the program does and what makes it change what it is doing. State of the code on 2026-10-11 (defaults fingerprint `b5651d39`).

The autopilot takes a winged vehicle (the target is the Mk3 shuttle `qs_shuttle2`) from orbit to a stop on the KSC runway. It does not follow a stored plan. Every tick it reads the vehicle's state, propagates the rest of the flight to the ground with an RK4 integrator (`trajectory.py`), and takes each decision from that fresh prediction. `autopilot.py` is the phase machine and the only code that talks to the game; `guidance.py` holds the control laws as pure functions; `environment.py` holds the aerodynamic table the vehicle measures for itself; every tunable is in `config.py`.

The phase machine

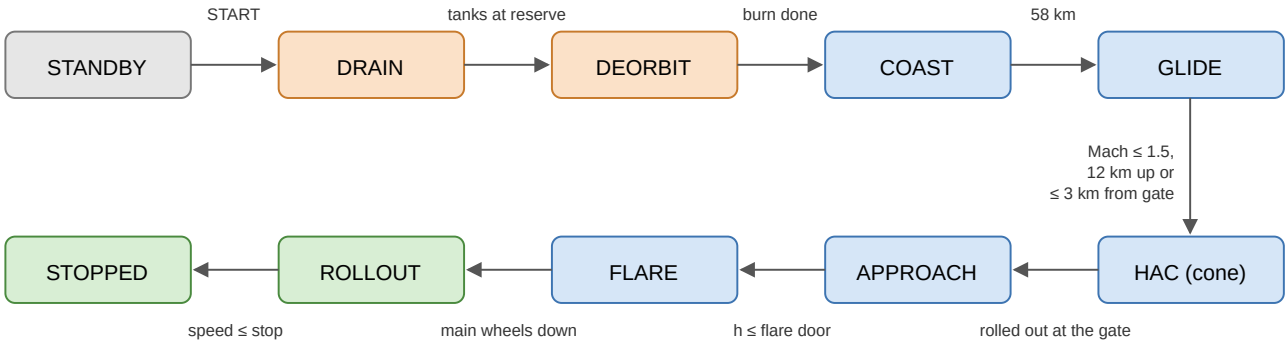


Figure 1. Phases and the conditions that advance them. Orange phases use the engine; blue are unpowered flight; green is on the ground. Engaged below the interface, START skips to GLIDE, APPROACH or FLARE by height and distance to the runway.

Things that run in every phase

mechanism	what it does, and when
Prediction	A propagation to the ground of the law the vehicle will fly (the same function the controller calls), from the current state, each tick. The deorbit and the glide solve against it; the cone and the approach plan from it.
Aerodynamic table	Swept in STANDBY with kRPC's `simulate_aerodynamic_force_at` over angle of attack and Mach; rows are re-probed in flight. Stall speed (an equivalent airspeed), best L/D and the approach speed are read from it, not configured.
Attitude	kRPC's autopilot points the nose at the commanded (alpha, bank). Its response times are derived from the vessel's inertia and torque and retuned as surface authority grows with dynamic pressure. A roll damper lengthens the roll response when bank swings grow and recovers it after.
RCS valve	Open when the pointing error exceeds 5°, closed 2 s after it is under 1.5°. Allowed in the burn, the coast, the glide above Mach 1 and during bank reversals.
Alpha ceiling	Learned: when the vehicle sits well below its commanded alpha, the ceiling ratchets down to what it holds, and the prediction uses the same limit (`trajectory.Holdable`, binned by dynamic pressure).
Pitch trim	Above Mach 3 in the glide the standing pitch input is moved off kRPC's integrators into a body-frame manual trim (`GLIDE_PITCH_OFFLOAD`). In the cone and approach the two nose canards are driven as a trim that walks the standing pitch input to zero (`CANARD_TRIM`, under 15° alpha, settled).
Brakes	Two built from the control surfaces, both probed in STANDBY: the split rudder (the two tail fins deployed in opposite directions: drag with no yaw or roll) and the spoiler (opposed flaps on the elevons). Both slew at a limited rate and are used only below the glide, in the order split rudder → spoiler → weave/S-turn.
Time scale	The farm's time-scale plugin is set so each tick's game interval stays what the phase asked for; every log ends with a `loop rate` line.

Phase by phase

STANDBY — until START is pressed

Configures the craft (main gear brakes 200% / friction 10, nose gear brake off), measures wheel clearance and tail-strike angle, tunes the attitude controller, sweeps the aerodynamic table, and finds the brakes it can build from the surfaces. On START: below the 58 km interface → GLIDE / APPROACH / FLARE by height and distance; otherwise → DRAIN.

DRAIN — before the burn

Opens the craft's drain valves and empties propellant down to what the burn needs, at the engine's vacuum Isp. Closes the valves → DEORBIT. Draining before the burn keeps every mass change upstream of the burn's own measurement.

DEORBIT — plan the window, line up, one burn

- **Plan.** On entry the orbit is propagated forward in vacuum and the burn search is asked of future states every 60 s (bisected to 8 s), up to three of Kerbin's days ahead and 1.5 s of work a tick. A pass qualifies when the runway is within the glide's reach along track (the arc from the interface to the gate at most 1400 km) and sideways (the gate's offset from the predicted ground track at most half the sideways reach of an entry leaning one way all the way down).
- **Cheap windows only.** A window qualifies only if its burn costs no more than the burn at apoapsis that puts the periapsis at the surface (`deorbit_dv_floor`, the burn the drain reserve is sized on). A dearer one is remembered and the scan skips half an orbit past it; if the horizon runs out, the cheapest seen is taken.
- **Search** (`deorbit_solution`): a grid then refinement over retrograde dv 10–400 m/s. For each candidate it propagates the two corners of what the glide may later command (`deorbit_window`): the shortest entry (high drag) and the longest (most lift). The burn chosen puts the gate 25% of the half-width short of the window's centre, the side the glide can always correct by shortening (Figure 2).
- **Wait.** A stock maneuver node is placed at the window; `warp_to` takes the game to 60 s before it (a learned rails warp is the fallback when no window was found). A window not seen 120 s after it was due is replanned.
- **The node carries the burn.** Surface retrograde × dv at the node's time, resolved into prograde, normal and radial (`guidance.node_components`); the sign of the normal component is read back once from the node's burn vector and logged as `burn node:`.
- **Line up.** kRPC's autopilot lets go and the game's SAS holds Maneuver (Retrograde if the core has no Maneuver mode), RCS on. The burn commits once the nose is within 3° of the burn vector on two ticks in a row, or after a hold stuck inside 8° for 6 s, or after 120 s.
- **Burn.** SAS switches to stability assist and holds the attitude it committed in. Thrust is limited so the burn lasts at least 3 s, throttle tapered on the dv still owed. The stop test re-propagates the arc each tick and measures the remaining dv against the chosen aim; once the dv owed reaches zero it runs even on ticks where the nose has drifted. When the propagation cannot answer (arrival past the 1500 s horizon) the burn stops on the solved dv and its exit line says `unmeasured`. At cutoff the node is removed, SAS off, the autopilot re-engaged → COAST.

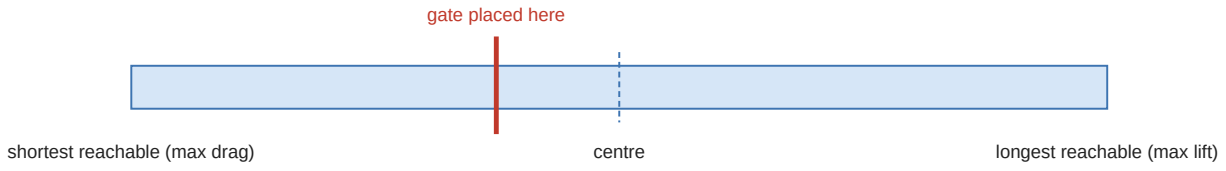


Figure 2. The deorbit window. The burn's dv moves the whole window; it is chosen so the gate sits 25% of the half-width short of centre.

COAST — to the 58 km interface

Swings to the entry attitude (22° alpha, 30° bank toward the runway) in vacuum, where holding it costs nothing, and `warp_to`s the part above the atmosphere (released 2 km above it). The bank's side is latched so the glide starts on the same lean. At 58 km → GLIDE.

GLIDE — the entry, Mach 25 to Mach 1.5

Each tick (1 s): solve alpha and bank against the prediction. What the solve nulls changes once, at Mach 4.5.

stretch	what the solve nulls
Hypersonic (above Mach 4.5)	Range: the along-track miss where the predicted arc crosses 12 km, aimed 500 m past the gate by a reserve that decays to nothing.
Terminal glide (Mach ≤ 4.5, latched)	The energy handed to the cone: the predicted energy height h + v²/2g where the arc meets the cone's entry test, against `HAC_ALT_M` (12 km) at the cone's gate speed plus a 1500 m margin (≈ 14.1 km). If the arc reaches 12 km before the entry circle, the path still to fly is priced at the cone's glide ratio and added; if that crossing is out of reach of the circle the miss is the shorter of the energy miss and the range shortfall, so a range short is never overridden by energy. The log's `long=` is then this energy surplus in metres, beside `tE=`.

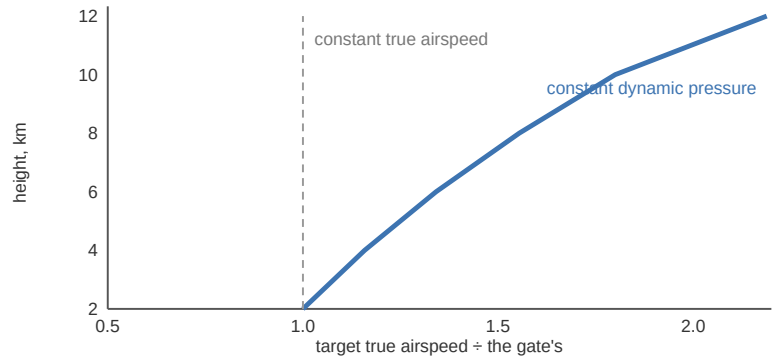
control	role
alpha (20–40°)	Downrange / energy: more alpha, more drag. Solved by sampling against the miss, rate-limited to 3°/s, held still during a bank reversal, clamped by speed and by the learned ceiling. In the terminal glide the prediction flies min(command, a 10 s mean of the achieved alpha + 1°), and while the predicted energy is in surplus the solve may only raise alpha.
bank magnitude (30–70°)	Takes what alpha leaves: rolling lift off vertical sinks the vehicle into denser air. When both are at their stops the solve returns the longest/shortest arc it can reach (`slv=max`).
bank sign	Cross-track: lean toward the runway; reverse when the angle between the ground track and the bearing to the gate leaves a deadband of 0.012° per km to run (0.5°–12°, never narrower than the heading turned in one roll time-to-peak), or the predicted cross-track leaves 20 m per km (500 m–40 km). A reversal commits once the commanded bank has passed 30° on the new side.

At Mach ≤ 1.5 and either 12 km over the field or within 3 km of the gate: drop the terminal-glide target, release the alpha ceiling for the landing, pick the runway end and turn direction that are cheaper from here (`hac_choose`) → HAC.

HAC — the heading-alignment cone

Each tick: re-plan the circle from the present state (`guidance.hac`).

- **Radius.** The circle (tangent to the extended centreline, ending at the gate) whose path, priced down a ladder of 500 m slices at the glide ratio the vehicle flies at each slice's target speed, spends the energy above the gate (height plus the speed above the gate's target). The plan prices the path at the braked L/D, so the brake is part of the plan. 2–16 km; a whole lap is added only when even the widest circle is short of path and the lap fits. The radius moves at a limited rate.
- **Speed: constant dynamic pressure** (`HAC_SPEED_Q`, Figure 3). The target is the speed whose dynamic pressure equals the gate target's (the approach speed, from the measured stall) at the height being flown: a constant equivalent airspeed, so the surfaces have the same authority and the same load needs the same alpha from cone entry to the handover. Alpha holds it: the trim alpha for the banked load plus 0.2° per m/s of excess, 0–22°, capped where lift reaches 3 g at the q being flown (`HAC_LOAD_MAX`).
- **Split rudder, first** (`HAC_SPLIT_BRAKE`). Each tick it solves the L/D factor that spends the height to the gate over the planned path, `path / ((h − gate) × clean L/D)`, and opens to the angle that gives it (`HAC_SPLIT_FACTOR`, 0–38°). It fades out only near the stall: across the 15% below 1.45 × the stall speed at the turn's load, in equivalent airspeed.
- **Spoiler, second.** Only once the split rudder is at its stop, opened in proportion to the surplus height, full at the weave's threshold (800 m). It deploys the roll surfaces, so it is pulled in whenever the bank is off its command.
- **Bank and weave, last.** The bank that flies the circle, capped at 45°, plus a weave (up to 50° off track) when more than 800 m of surplus height remains and the split rudder is at its stop or faded out.
- **Exit.** Lined up (within 12°), within 900 m of the gate, no lap left, and no more than the approach can absorb above its need → APPROACH. Below the gate's 2 km → APPROACH regardless.



At 12 km a constant true airspeed equal to the gate's is below the stall in equivalent terms: holding 1 g there needs the top of the lift curve. At constant dynamic pressure the same load needs the same, low, alpha at every height, and the true speed falls by itself as the air thickens; that kinetic energy is in the plan's budget, which counts speed above the gate's target as height.

Figure 3. The cone's speed target against height (Kerbin's density; about 112 m/s true at the 2 km gate and 245 m/s at 12 km on the shuttle).

APPROACH — final, 2 km to the flare

- **Aim point**: 1800 m past the threshold (the approach aims 1000 m nearer, so the flare's float lands in the zone).
- **Speed, not path**: alpha is set from the descent angle that holds the target speed (2.25 × stall), from dv/dt = g sinθ − D/m, so it can make speed by steepening as well as lose it.
- **Split rudder** (`APPROACH_SPLIT_BRAKE`): opens to the L/D factor the geometry asks for, `distance / (height × best L/D)`, at 10°/s. Over 1.5 × the design sink it fades across the next 30% of that cap and holds while faster than its target speed; under 1.45 × stall it fades across the 15% below. It is in 4 s of sink above the flare door.
- **Spoiler**: once the split rudder is at its stop, in proportion to the excess height, full at the S-turn's threshold; stowed with the rudder, pulled in whenever the bank is off its command.
- **Lateral**: a closure that brings cross-track to zero as the wheels arrive; S-turns (up to 45° of track) when more than 200 m of excess height remains, the aim is over 4 km away and both brakes are at their stops.
- **Flare door**: 50 m + 2.5 s × sink rate. At or below it → FLARE.

FLARE — the last 50–100 m

Stows the brakes. Tracks a sink rate of $\sqrt{(v_{td}^2 + 2\,a\,h)}$ that eases to the touchdown rate, with alpha capped by maximum lift (30°) and by the tail-strike angle. Above 60 m it may bank up to 12° (tapering to wings level) to close cross-track; below 30 m the nose is aligned with the runway, not the airflow. Gear down. Main wheels touching → ROLLOUT.

ROLLOUT — on the runway

Ground spoiler out on main-gear contact. The nose is ramped down from its touchdown attitude over 1.5 s, the trims ramp out, full main-gear braking, nosewheel steering onto the centreline on cross-track plus its rate times a 3 s lead. At the stop speed → STOPPED, which logs the stop position and hands the vessel back.

Which law decides what

quantity	GLIDE	HAC	APPROACH	FLARE / ROLLOUT
downrange / energy	alpha, then bank magnitude — on range above Mach 4.5, on cone-entry energy below	circle radius and laps; split rudder → spoiler → weave	split rudder → spoiler → S-turns	—
speed	speed cap on alpha (late)	alpha to a constant dynamic pressure	descent angle that holds speed	sink-rate law / wheel brakes
cross-track	bank sign, azimuth and cross-track deadbands	circle geometry	closure, wheels-time	bank ≤ 12°, then nosewheel
pitch trim	body-frame offload (Mach > 3)	canard trim	canard trim	ramped out

Logs

Nothing prints. Each run writes `logs/LOG<n>`: a `config:` line (fields different from the defaults, plus the defaults fingerprint), one telemetry line per tick interval, every phase change and decision as an event (each brake's out/in with its reason, the burn node's components, the terminal glide's start and its target), and a closing `loop rate` line. The DOWN line gives the stop along and across the runway and how far north or south of the runway line it is (the space center lies to the south). `spaceplane/tools/` reads them (`landsum.py`, `armsum.py`, `conesum.py`, `oscsum.py`).